

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE

COURSE CODE: CSA1221

Name: A Althaf

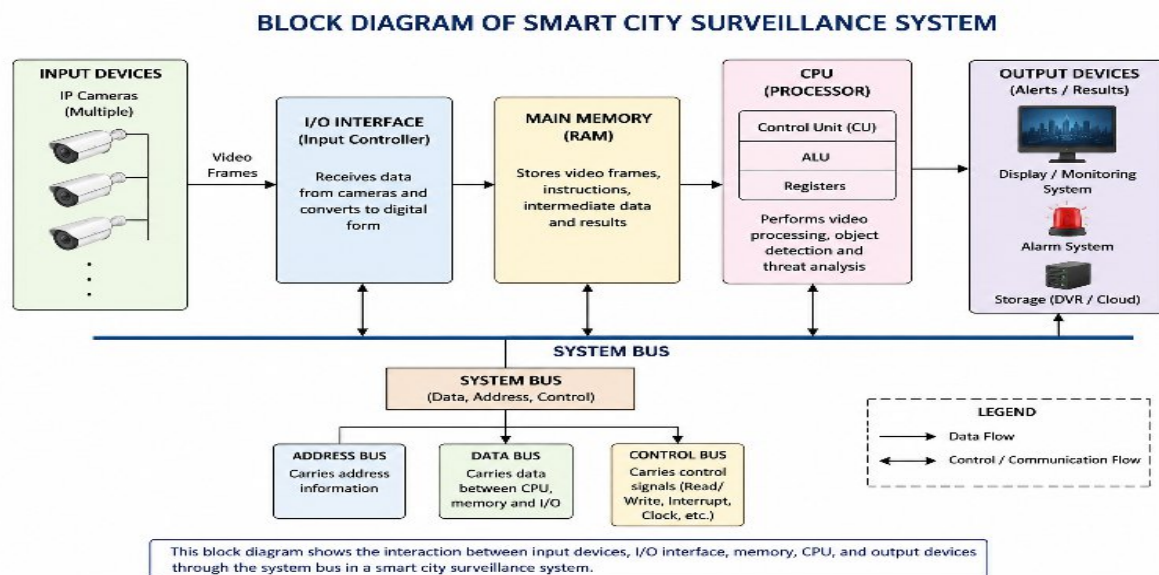
Reg.no:192521257

1.A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.

Introduction

Modern smart cities use surveillance systems with multiple high-definition cameras to improve public safety. Every second, thousands of video frames are transferred from cameras to the processor for object recognition and threat detection. During peak traffic hours, the amount of incoming data increases significantly, causing delays in processing. The main reason is the heavy interaction between CPU, memory, and I/O devices, which affects overall system performance.

System Block Diagram



Analysis of Functional Units

CPU

The CPU performs image analysis, object classification, and decision-making. It continuously fetches instructions from memory, processes video frames, and generates security alerts. When many cameras operate simultaneously, the CPU becomes overloaded because it must process a large number of instructions every second.

Memory

RAM temporarily stores captured video frames and program instructions. If the memory bandwidth is insufficient, the CPU must wait until data becomes available. This waiting time increases execution delay.

Input/Output Devices

The cameras act as input devices, while storage devices and alarm systems act as output devices. Continuous data transfer from multiple cameras generates high I/O traffic, increasing bus utilization.

Effect of Bus Structures

Single-Bus Architecture

In a single-bus system, the CPU, memory, and I/O devices share one communication path.

Impact

- High bus contention
- Increased waiting time
- Lower throughput
- Delayed threat detection

Multi-Bus Architecture

A multi-bus system provides separate buses for memory and I/O communication.

Advantages

- Parallel data transfer
- Reduced waiting time
- Higher bandwidth
- Improved processor efficiency
- Faster response during emergencies

Instruction Execution Cycle Analysis

Each instruction passes through the following stages:

1. Fetch
2. Decode
3. Execute
4. Memory Access
5. Write Back

When memory becomes busy, the fetch stage is delayed, increasing the total execution time.

Performance Improvement Techniques

Pipeline processing executes multiple instructions simultaneously.

Cache memory reduces RAM access time.

Direct Memory Access (DMA) transfers video directly into memory without CPU intervention.

Multi-core processors distribute camera streams among different cores.

Conclusion

The performance bottleneck mainly occurs because CPU, memory, and I/O devices compete for the same communication resources. A multi-bus architecture combined with pipelining, DMA, and cache memory greatly improves system throughput and reduces surveillance delay.

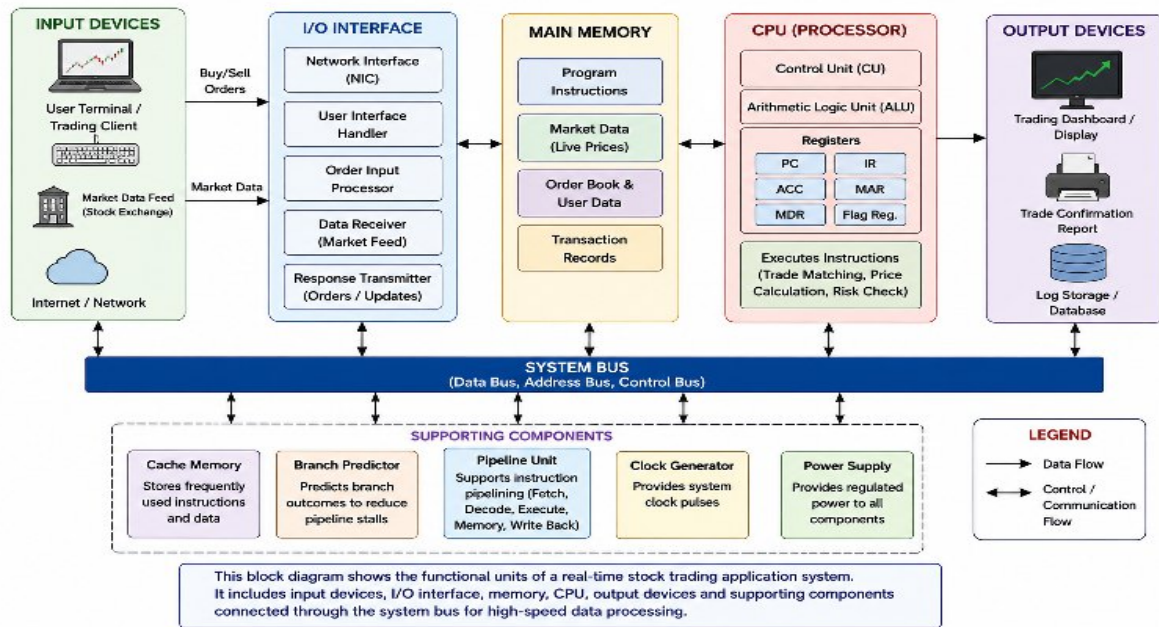
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.

Introduction

Stock trading systems execute millions of financial transactions within milliseconds. During market opening and closing hours, users experience transaction delays because the processor spends additional cycles accessing memory and resolving branch instructions.

System Block Diagram

Block Diagram of Real-Time Stock Trading Application System



Instruction Execution Cycle Analysis

Every transaction follows the Fetch–Decode–Execute cycle.

Fetch: Instruction retrieved from memory.

Decode: Processor interprets the instruction.

Execute: Arithmetic or logical operation performed.

Memory Access: Reads or updates trading records.

Write Back: Stores the result.

Frequent memory accesses and branch instructions increase processor waiting time.

Performance Metrics

CPU performance is determined by

Higher CPI directly increases execution time.

Reasons for High CPI

Frequent cache misses

Large database access

Conditional branch instructions

Pipeline interruptions

Memory latency

Architectural Enhancements

Cache Memory

Frequently accessed customer data remains inside cache memory, reducing memory latency.

Branch Prediction

Predicts branch outcomes before execution, minimizing pipeline stalls.

Superscalar Processor

Executes multiple instructions in one clock cycle.

Out-of-Order Execution

Independent instructions execute without waiting for previous stalled instructions.

Multi-Core Processing

Different processor cores execute different customer requests simultaneously.

Analysis

Reducing CPI has a greater impact than simply increasing clock frequency because lower CPI decreases execution time without increasing power consumption significantly.

Conclusion

Cache optimization, branch prediction, superscalar execution, and multi-core architecture effectively reduce CPI and improve transaction speed during high

trading loads.

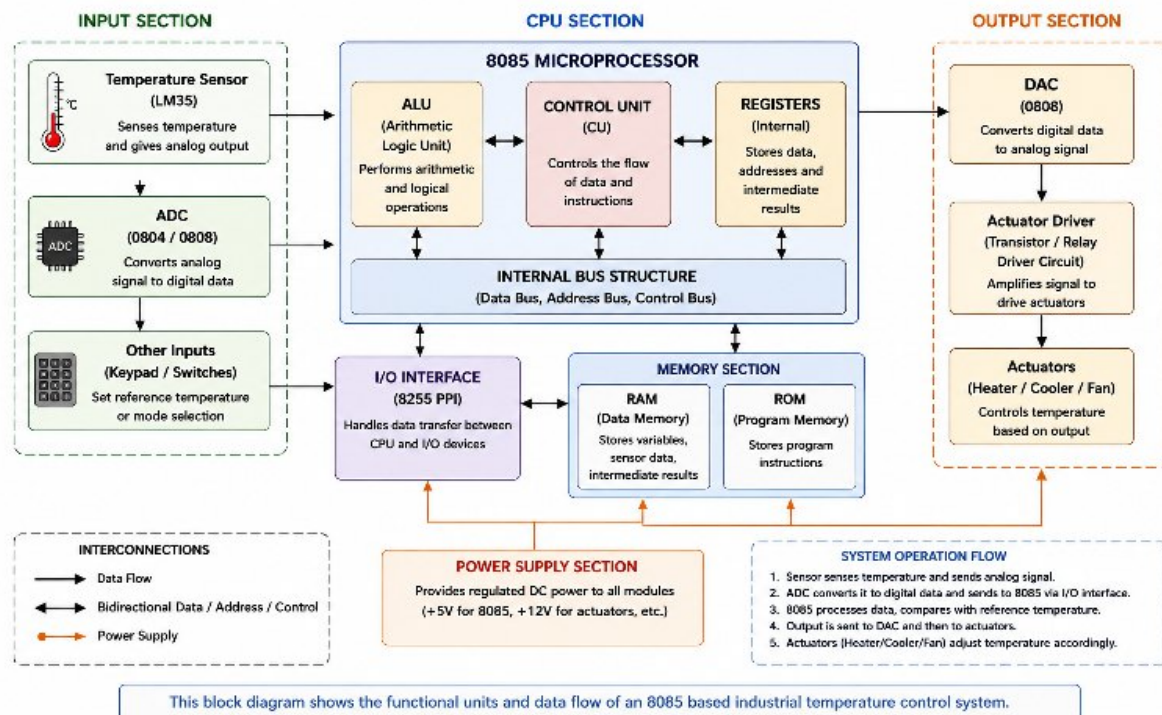
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

Introduction

An 8085-based industrial temperature control system is used to monitor and regulate temperature in industrial applications. The temperature sensor continuously measures the temperature and sends the data to the 8085 microprocessor through an ADC. The processor compares the measured value with the reference temperature and controls the heater or cooling fan accordingly. Proper addressing modes ensure that the processor accesses the correct memory location, improving system accuracy and reliability.

System Block Diagram

BLOCK DIAGRAM – 8085 BASED INDUSTRIAL TEMPERATURE CONTROL SYSTEM



Influence of Addressing Modes

Immediate Addressing: Stores constant values such as reference or threshold temperatures directly in the instruction.

Register Addressing: Transfers data between processor registers, providing fast execution with minimum memory access.

Direct Addressing: Accesses sensor data stored at a fixed memory location, making the program simple and easy to understand.

Register Indirect Addressing: Uses the HL register pair to access different memory locations dynamically, making it suitable for continuously changing sensor data.

Possible Causes of Errors

Incorrect temperature readings may occur due to:

Wrong memory address selection

Incorrect HL register initialization

Corrupted sensor data

Register overwrite during program execution

Improper memory or I/O initialization

These errors can lead to inaccurate temperature control and unstable system operation.

Effective Use of the 8085 Instruction Set

The following instructions are commonly used:

LDA – Load sensor data from memory

STA – Store processed data

MOV – Transfer data between registers

MVI – Load immediate values

CMP – Compare measured and reference temperatures

ANI, ORA – Perform logical operations

JZ, JC, JMP – Execute conditional and unconditional branching

CALL, RET – Execute and return from subroutines

These instructions help the processor perform accurate sensing, comparison, and actuator control.

Analysis

Selecting the correct addressing mode improves memory access efficiency and reduces processing errors. Register Indirect Addressing is particularly useful because it allows the processor to access changing sensor locations without modifying the program. Efficient instruction execution also improves the speed and reliability of temperature control.

Conclusion

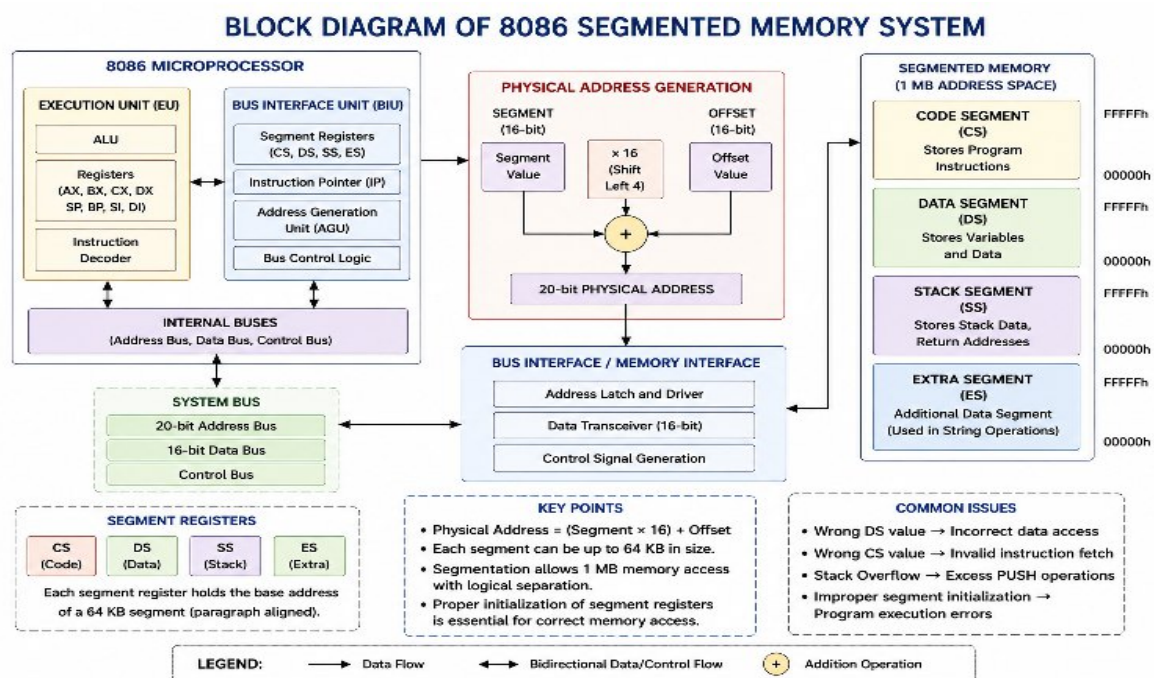
The performance of an 8085-based industrial temperature control system depends on the proper use of addressing modes and instruction execution. Correct memory access and efficient use of the 8085 instruction set ensure accurate temperature

measurement, reliable control of heaters and cooling devices, and stable industrial automation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.

Introduction

The 8086 microprocessor uses a segmented memory architecture to access up to 1 MB of memory. Memory is divided into logical segments such as Code Segment (CS), Data Segment (DS), Stack Segment (SS), and Extra Segment (ES). Proper management of these segments ensures correct program execution, while improper segment handling can lead to incorrect data access and stack-related errors.



Working of Segmentation

The physical address in the 8086 is calculated using:

The four memory segments are:

Code Segment (CS): Stores program instructions.

Data Segment (DS): Stores variables and data.

Stack Segment (SS): Stores return addresses and temporary data.

Extra Segment (ES): Stores additional data for string operations.

Analysis of Problems

Common problems caused by improper segmentation include:

Incorrect Data Access: Wrong initialization of the **DS register** causes the processor to read incorrect memory locations.

Stack Overflow: Excessive **PUSH** operations without corresponding **POP** operations exceed stack capacity.

Program Execution Errors: Incorrect **CS** values cause the processor to fetch invalid instructions, leading to unexpected program behavior.

Managing Instruction Execution

Reliable program execution can be achieved by:

Initializing all segment registers correctly.

Loading the appropriate data segment before execution.

Maintaining balanced **PUSH** and **POP** instructions.

Verifying the stack pointer before stack operations.

Addressing Modes

The **8086** supports the following addressing modes:

Immediate

Register

Direct

Based

Indexed

Based Indexed

Proper selection of addressing modes improves memory access efficiency and reduces programming errors.

Analysis

Memory segmentation allows the 8086 processor to organize programs efficiently and support larger applications. Correct initialization of segment registers and proper use of addressing modes ensure accurate memory access, efficient instruction execution, and reliable multitasking performance.

Conclusion

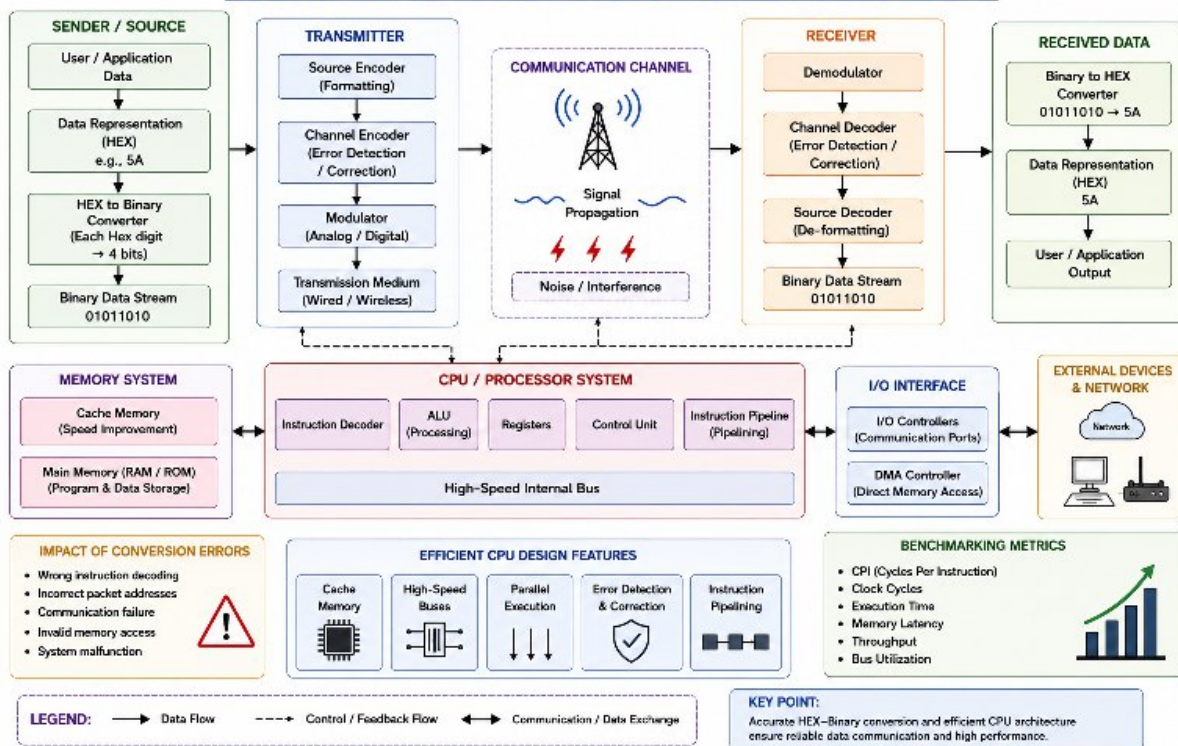
Proper memory segmentation and correct addressing techniques improve the reliability, efficiency, and stability of the 8086 microprocessor. Careful management of segment registers and stack operations prevents memory access errors and stack overflow, ensuring accurate program execution in real-time applications.

6.A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

Introduction

In data communication systems, information is commonly represented in hexadecimal format because it is compact and easy to interpret. Before processing, the processor converts hexadecimal values into binary, which is the language understood by digital hardware. Accurate number system conversion is essential to ensure reliable data transmission and processing.

BLOCK DIAGRAM OF DATA COMMUNICATION SYSTEM



Importance of Number System Conversion

Hexadecimal simplifies debugging and reduces the length of binary values while maintaining accuracy.

Example:

Hexadecimal: 5A

Binary: 0101 1010

Each hexadecimal digit represents **4 binary bits**, making conversion simple and efficient.

Impact of Conversion Errors

Incorrect hexadecimal-to-binary conversion may cause:

Wrong instruction decoding

Incorrect packet addresses

Communication failure

Invalid memory access

System malfunction

Even a single-bit error can change the meaning of data, leading to transmission errors.

Role of Efficient CPU Design

Modern processors improve communication reliability by using:

Cache memory

High-speed bus architecture

Parallel execution

Error detection and correction mechanisms

Instruction pipelining

These features enhance processing speed and minimize communication errors.

Benchmarking

Benchmarking evaluates processor performance using:

CPI (Cycles Per Instruction)

Clock Cycles

Execution Time

Memory Latency

Throughput

Bus Utilization

These metrics help identify performance bottlenecks and optimize communication systems.

Analysis

Accurate hexadecimal-to-binary conversion is critical for correct instruction execution and reliable data transmission. Combining efficient CPU architecture with benchmarking improves system performance, reduces errors, and enhances the reliability of real-time communication systems.

Conclusion

Accurate hexadecimal-to-binary conversion is essential for reliable data communication, as even a single-bit error can result in incorrect instruction execution and transmission failures. Efficient CPU features such as cache memory, high-speed buses, pipelining, and error detection mechanisms improve communication performance and system reliability. Additionally, benchmarking metrics like CPI, execution time, memory latency, and throughput help evaluate and optimize processor performance, ensuring fast and error-free data communication.